CCSU BCA | SEMESTER I

Computer Architecture

Digital logic, CPU organization, input/output and memory

COURSE CODE	BCA-1002T | Archive label: 19008
SEMESTER	BCA Semester I
SYLLABUS	CCSU BCA 2025-26 / NEP

The current course joins digital logic with computer organization. Learn the circuit truth tables first, then follow how instructions, data and control move through a computer.

Targeting 80+ is a study goal, not a guaranteed result. Use these notes with class material, the current syllabus and timed paper practice.

How to use this guide

Learn the short unit summary first. Then answer each question without looking, compare your answer with the model, and rewrite it in your own words. For calculations and programs, write every step and test with a small example.

Contents

- Unit I | Digital principles, number systems and K-maps
- Unit II | Combinational and sequential circuits
- Unit III | Basic computer organization and CPU
- Unit IV | Pipeline, input/output, arithmetic and memory
- Question-paper coverage and pattern
- Last-five-year archive check
- Learning resources and references

Unit I | Digital principles, number systems and K-maps

Digital systems use discrete values, commonly 0 and 1. Boolean algebra, number conversion and Karnaugh maps provide compact ways to reason about and simplify circuits.

Core ideas to remember

- A digital signal represents a finite set of levels; binary logic uses 0 and 1.
- Von Neumann architecture stores instructions and data in memory and uses a processor to fetch and execute instructions.
- K-map groups adjacent 1-cells for SOP or 0-cells for POS. Group sizes are powers of two.
- Binary, octal and hexadecimal conversions use positional weights; group binary digits by 3 for octal and 4 for hexadecimal.
- Gray code changes one bit between adjacent values; ASCII and EBCDIC encode characters; parity can detect some errors.

Important questions with model answers

Question: Simplify F(A,B,C)=sum m(1,3,5,7) using a K-map.

Model answer

Place 1s in minterms 1, 3, 5 and 7. These are all combinations where C=1, regardless of A and B. Group the four adjacent cells as one group. A and B vary within the group and cancel; C remains. Therefore F=C. In a full exam answer, draw the correctly labelled K-map, show the group and state the simplified result.

Question: Convert (101101.01)2 to decimal.

Model answer

Use positional weights. The integer part is 1x32 + 0x16 + 1x8 + 1x4 + 0x2 + 1x1 = 45. The fractional part is 0x1/2 + 1x1/4 = 0.25. Therefore (101101.01)2 = 45.25 in decimal. For hexadecimal, group as 0010 1101 . 0100, giving 2D.4 in base 16.

Unit II | Combinational and sequential circuits

Combinational outputs depend on current inputs. Sequential outputs also depend on stored state. Distinguish their use before drawing a circuit.

Core ideas to remember

- A half adder adds two bits: Sum=A XOR B; Carry=A AND B.
- A full adder adds A, B and carry-in. A multiplexer selects one input; a decoder activates an output based on an input code.
- An SR, D, JK or T flip-flop stores one bit; the clock and input determine its next state.
- Registers hold multiple bits. Shift registers move bits left or right; counters advance through states.
- Synchronous counters share a clock; asynchronous counters pass transitions from one stage to the next.

Important questions with model answers

Question: Compare a half adder and a full adder.

Model answer

A half adder adds two one-bit inputs and produces sum and carry. It has no carry-in input, so it cannot directly handle the carry from a previous stage. A full adder adds A, B and carry-in, producing sum and carry-out. Full adders can be connected in a chain to add multi-bit binary numbers.

Question: What is the difference between combinational and sequential logic?

Model answer

In combinational logic, output is a function of the current input only; examples include decoders and adders. Sequential logic includes memory or state, so output can depend on current input and the previous state; flip-flops and counters are examples. When explaining a sequential circuit, identify its state variable, clock behaviour and next-state rule.

Unit III | Basic computer organization and CPU

A CPU executes instructions by moving values among registers, memory and functional units. An instruction cycle gives a useful frame for explaining this sequence.

Core ideas to remember

- Common registers include the program counter, instruction register, memory address register and accumulator.
- The fetch-decode-execute cycle fetches an instruction, decodes its operation and operands, executes it and updates the program counter or state.
- Addressing modes describe how an instruction locates its operand, such as immediate, direct, indirect or register addressing.
- RISC designs favour a simpler instruction set and regular formats; CISC designs provide more complex instructions. Real processors combine design choices.
- Stack organization uses last-in, first-out storage for return addresses, temporary values or parameters.

Important questions with model answers

Question: Describe the instruction cycle.

Model answer

The program counter identifies the next instruction. The control unit sends a memory-read request and the instruction is loaded into the instruction register; the program counter advances. The control unit decodes the instruction, identifies the operation and operands, then directs the required register, ALU or memory action. Finally, results are stored and control moves to the next instruction or to a branch target. An interrupt may temporarily redirect execution.

Question: Compare RISC and CISC at an introductory level.

Model answer

RISC traditionally emphasizes a smaller set of simpler instructions, regular instruction formats and efficient pipelining. CISC traditionally provides a larger set of more complex instructions, sometimes with variable formats and operations that combine several steps. The distinction is a design tendency rather than a strict rule: modern processors may use complex decoding internally while executing simpler micro-operations.

Unit IV | Pipeline, input/output, arithmetic and memory

The final unit connects performance and data transfer. Cache, virtual memory, interrupts, DMA and pipelining all address the gap between processor speed and the time needed to access resources.

Core ideas to remember

- A pipeline overlaps instruction stages. Ideal speedup is limited by hazards, unequal stage time and pipeline startup or drain.
- Interrupt-driven I/O lets a device request attention; DMA transfers a block between device and memory with less CPU involvement.
- Booth's algorithm handles signed binary multiplication by examining adjacent multiplier bits and applying add, subtract or shift operations.
- Memory hierarchy places small, fast storage near the CPU and larger, slower storage farther away.
- Cache exploits locality. Virtual memory gives processes a logical address space backed by physical memory and secondary storage.

Important questions with model answers

Question: Explain cache memory and locality.

Model answer

Cache is a small, fast memory between the CPU and main memory. It stores recently or frequently used data and instructions. Temporal locality means recently used items may be used again; spatial locality means nearby addresses may be used soon. A cache hit supplies data quickly; a miss requires a slower lower-level access. Explain hit rate, blocks and replacement policy when the question asks for detail.

Question: Compare programmed I/O, interrupt-driven I/O and DMA.

Model answer

With programmed I/O, the CPU repeatedly checks device status and transfers data. Interrupt-driven I/O lets the device signal the CPU when it needs service, reducing busy waiting. DMA uses a controller to move a block between a device and memory; the CPU sets up the transfer and is notified when it completes. DMA is efficient for larger transfers, while interrupts suit events needing CPU attention.

Paper pattern and topic analysis

The CCSU syllabus assigns the theory course 25 internal plus 75 external marks and provides a separate practical assessment. The archive lists a December 2025 BCA Semester I paper (label 19008), but its full question text could not be independently inspected here. Do not assume an unverified section pattern.

The current course has four units and explicitly spans digital principles, circuits, CPU, pipeline, I/O and memory. Related archived courses from earlier semesters are only partial analogues: Digital Electronics and Computer Organisation shares logic and sequential-circuit topics; Computer Architecture and Assembly Language shares instruction cycles, Booth multiplication, I/O and cache. A 2024 BCA-303 paper visibly tests several latter topics. These are useful overlap signals, not evidence that the current Semester I paper repeats the same questions.

What to prioritize

- Practise number conversions and K-map simplification with clearly labelled maps.
- Draw truth tables and explain adders, multiplexers, flip-flops, registers and counters.
- Learn register roles, instruction cycle, addressing modes and RISC/CISC comparison.
- Prepare cache, virtual memory, pipeline, interrupts, DMA and Booth multiplication as long answers.

Last-five-year archive check

A current BCA Semester I Computer Architecture paper is listed for 2025 (archive label 19008). The archive also has earlier BCA Semester II Digital Electronics and Computer Organisation papers and BCA Semester III Computer Architecture and Assembly Language papers, including 2021-2025 entries. They are not the same course or semester. They are cited only for topic overlap; no five-year recurrence claim is made for BCA-1002T.

How to read the evidence

The current unit map comes from the 2025-26 CCSU syllabus reproduction. Older-paper overlap is checked against the archive listings and a 2024 BCA-303 paper. The exact BCA-1002T 2025 questions and format remain unverified in the accessible text, so the priority list is syllabus-led.

Learning resources and source list

The paper archive is a discovery aid, not the university's official syllabus. The current course titles and units below follow the 2025-26 CCSU syllabus reproduced by an affiliated college. Older paper titles are identified as legacy or related material wherever they differ.

CCSU BCA syllabus and unit list (2025-26 reproduction)

CCSUStudy BCA question-paper archive

December 2025 Computer Architecture paper listing

December 2024 BCA-303 related paper listing

NPTEL Computer Organization and Architecture, IIT Guwahati

Mano, Computer System Architecture (listed in syllabus readings)

Coverage note: no question bank can predict the exact paper. Treat the priority list as a revision map and prepare every listed unit.